

Relatório de Prática: Conexão com API (Prática 2)

1. Introdução

Este relatório descreve as atividades desenvolvidas durante a Prática 2, cujo objetivo principal foi estabelecer a integração entre uma aplicação cliente e uma API (Application Programming Interface). A conectividade via API é a base do desenvolvimento web moderno, permitindo a comunicação eficiente e segura entre sistemas heterogêneos.

2. Objetivos

- Compreender o funcionamento do protocolo HTTP/HTTPS.
- Manipular métodos de requisição essenciais (GET, POST, PUT, DELETE).
- Tratar respostas estruturadas em formato JSON.
- Gerenciar autenticação e chaves de API (API Keys), se aplicável.
- Implementar o tratamento de erros em requisições assíncronas.

3. Ferramentas e Tecnologias Utilizadas

- **Linguagem de Programação:** JavaScript / TypeScript / Python
- **Ambiente de Execução:** Node.js / Navegador Web
- **Biblioteca/Client HTTP:** Fetch API / Axios / Postman
- **API Utilizada:** JSONPlaceholder / API Pública / API Própria

4. Desenvolvimento e Implementação

4.1. Configuração do Ambiente

O ambiente de desenvolvimento foi devidamente configurado por meio da inicialização do projeto e da instalação das dependências necessárias para gerenciar as chamadas de rede e o fluxo de dados.

4.2. Estrutura do Código

O código fonte foi modularizado com o objetivo de separar de forma clara a lógica da interface gráfica da camada de consumo de dados da API. Abaixo está representado o trecho principal da implementação da conexão:

```
// Exemplo de código utilizando Fetch API
async function buscarDadosDaAPI() {
  const url = 'https://api.exemplo.com/dados';

  try {
    const response = await fetch(url, {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': 'Bearer SEU_TOKEN_AQUI'
      }
    });
  }
};
```

```
    if (!response.ok) {  
      throw new Error(`Erro na requisição: ${response.status}`);  
    }  
  
    const data = await response.json();  
    console.log('Dados recebidos com sucesso:', data);  
    return data;  
  } catch (error) {  
    console.error('Falha na conexão com a API:', error);  
  }  
}
```

4.3. Tratamento de Respostas e Erros

- **Status 200/201:** Requisições bem-sucedidas. Os dados foram capturados e devidamente renderizados na tela.
- **Status 400/404:** Erros no cliente. Tratados exibindo notificações informativas ou mensagens de 'Recurso não encontrado'.
- **Status 500:** Erros no servidor. Tratados por meio de alertas indicando instabilidade momentânea no sistema externo.

5. Resultados Obtidos

A aplicação cliente foi capaz de disparar as requisições HTTP mapeadas de forma consistente. Os dados retornados pelo servidor em formato JSON foram capturados, validados e integrados à interface do usuário em tempo real, comprovando o correto funcionamento do fluxo ponta a ponta.

6. Conclusão

Esta atividade prática permitiu consolidar de forma clara os conceitos de assincronismo (Promises e Async/Await) e gerenciamento de fluxos de rede na internet. A arquitetura de conexão utilizando o API Connect demonstrou ser estável, performática e aderente aos requisitos técnicos estipulados para o projeto.